Parcial 2

Algoritmos evolutivos, genéticos, ascenso de colinas,
temple simulado, ACO y PSO

Nombre: _____ **Fecha:** _____

Duración sugerida: 2 días. **Calificación total:** 5,0 (cinco puntos de 1,0).

Objetivo: evaluar la comprensión de la representación de soluciones, las reglas de actualización y los mecanismos de exploración y explotación de algoritmos genéticos, ascenso de colinas, temple simulado, ACO y PSO.

Punto 1. Algoritmos evolutivos y algoritmos genéticos — 1,0

Se desea maximizar:

$$f(x) = x^2, \qquad x \in \{0, 1, \ldots, 31\}.$$

Cada solución se representa mediante un cromosoma binario de cinco bits. La población inicial es:

$$P = \{00101,\ 01100,\ 10010,\ 11001\}.$$

A. Componente matemático y conceptual — 0,5

1. Decodifique los cuatro cromosomas y calcule su aptitud.
2. Calcule la probabilidad de selección por ruleta de cada individuo:

$$p_i = \frac{f(x_i)}{\sum_j f(x_j)}.$$

3. Se seleccionan los padres `01100` y `11001`. Realice un cruce de un punto **después del segundo bit** y determine los dos descendientes.
4. En el primer descendiente, invierta el **último bit**. Determine su nuevo valor decimal y su aptitud.
5. Explique brevemente:
   - ¿Qué relación existe entre un algoritmo evolutivo y un algoritmo genético?
   - ¿Qué función cumplen selección, cruce y mutación?
   - ¿Por qué una mutación puede ser útil aunque empeore la aptitud inmediata?

B. Implementación en Python — 0,5

Implemente un algoritmo genético para este problema con:

- Población inicial: los cuatro cromosomas dados.
- Selección por ruleta.
- Cruce de un punto, con probabilidad $0,8$.

- Mutación independiente por bit, con probabilidad $0,05$.
- Elitismo: conservar el mejor individuo de la generación.
- Población constante de cuatro individuos.
- Veinte generaciones y semilla `42`.

Entregue: código, mejor cromosoma final, valor de x, aptitud e historial de la mejor aptitud por generación.

Pregunta de interpretación: ¿el algoritmo encontró el óptimo global? Justifique comparando con el máximo posible de la función. Explique por qué una ejecución de veinte generaciones no garantiza encontrarlo.

Punto 2. Ascenso de colinas y óptimos locales — 1,0

Considere el siguiente espacio discreto:

x	0	1	2	3	4	5	6	7	8
$f(x)$	1	4	7	5	3	6	9	8	5

Se desea maximizar $f(x)$. Los vecinos de x son $x-1$ y $x+1$, siempre que pertenezcan al dominio. El algoritmo elige el vecino con mayor valor y solo se mueve si existe una **mejora estricta**.

A. Componente matemático y conceptual — 0,5

1. Ejecute manualmente ascenso de colinas desde $x_0 = 0$. Registre los estados visitados.
2. Repita desde $x_0 = 4$.
3. Identifique los máximos locales y el máximo global.
4. Explique por qué cambiar el estado inicial puede modificar el resultado.
5. Proponga una estrategia para reducir el riesgo de terminar en un máximo local y explique su costo adicional.

B. Implementación en Python — 0,5

Implemente una función con la siguiente interfaz:

```
def ascenso_colinas(valores, inicio):
    """Devuelve el estado final y la trayectoria."""
```

Ejecute la función desde **todos los estados iniciales posibles**, de 0 a 8.

Entregue: una tabla con estado inicial, trayectoria, estado final y valor final.

Pregunta de interpretación: ¿qué proporción de los nueve estados iniciales conduce al máximo global? Explique cómo se relaciona este resultado con los reinicios aleatorios.

Punto 3. Temple simulado: aceptación de soluciones peores — 1,0

Se desea minimizar:

$$E(x) = x^2, \qquad x \in \{-5, -4, \ldots, 5\}.$$

La regla de aceptación es:

$$P(\text{aceptar}) = \begin{cases} 1, & \Delta E \leq 0, \\ e^{-\Delta E/T}, & \Delta E > 0, \end{cases} \qquad \Delta E = E(x_{\text{nuevo}}) - E(x_{\text{actual}}).$$

Una solución peor se acepta cuando $u < P(\text{aceptar})$, con u uniforme en $[0, 1)$.

A. Componente matemático y conceptual — 0,5

Analice **cada transición de forma independiente**:

Caso	Estado actual	Candidato	T	u
A	3	2	5	0,70
B	2	3	5	0,30
C	2	3	1	0,30

1. Calcule ΔE, la probabilidad de aceptación y la decisión en cada caso.
2. Explique cómo influye la temperatura en la aceptación de soluciones peores.
3. ¿Qué dificultad puede producir un enfriamiento demasiado rápido?
4. Compare la regla de movimiento de temple simulado con la de ascenso de colinas.

B. Implementación en Python — 0,5

Implemente temple simulado con:

- Estado inicial $x_0 = 5$.
- Temperatura inicial $T_0 = 10$.
- Enfriamiento geométrico $T_{k+1} = 0,90T_k$.
- Candidato: escoger aleatoriamente uno de los vecinos válidos $x - 1$ o $x + 1$.
- Cien iteraciones y semilla `42`.
- Registro del estado actual y de la mejor solución encontrada.

Entregue: código, mejor solución, energía mínima e historial de temperatura y energía.

Pregunta de interpretación: ¿por qué conviene almacenar la mejor solución encontrada, además del estado actual?

Punto 4. ACO: elección de rutas y actualización de feromonas — 1,0

Una hormiga se encuentra en el nodo A y debe elegir su siguiente nodo. Todos los destinos de la tabla están disponibles:

Arista	Distancia d_{Aj}	Feromona τ_{Aj}
$A \to B$	2	1
$A \to C$	4	2
$A \to D$	5	1

La información heurística y la probabilidad de elección son:

$$\eta_{Aj} = \frac{1}{d_{Aj}}, \qquad p_{Aj} = \frac{\tau_{Aj}^{\alpha}\eta_{Aj}^{\beta}}{\sum_{k\in\{B,C,D\}} \tau_{Ak}^{\alpha}\eta_{Ak}^{\beta}},$$

con $\alpha = 1$ y $\beta = 2$.

A. Componente matemático y conceptual — 0,5

1. Calcule el peso y la probabilidad de cada alternativa.
2. Identifique la alternativa más probable. ¿Su elección está garantizada?
3. Una hormiga completa una ruta de longitud $L = 10$ que utiliza $A \to B$. Actualice las tres feromonas mediante:
$$\tau_{Aj}^{\text{nueva}} = (1-\rho)\tau_{Aj} + \Delta\tau_{Aj}, \qquad \rho = 0,2,$$
donde $Q = 10$, $\Delta\tau_{AB} = Q/L$ y el depósito en las otras dos aristas es cero.
4. Explique la función de la evaporación y qué podría ocurrir si la feromona dominara excesivamente la elección.

B. Implementación en Python — 0,5

Implemente un **modelo simplificado de ACO para elegir entre tres rutas completas** de A a Z:

Ruta	Longitud total
$A - B - Z$	6
$A - C - Z$	8
$A - D - Z$	10

En este modelo, cada ruta tiene una única feromona τ_r, inicialmente igual a 1. Use:

$$p_r = \frac{\tau_r^{\alpha}(1/L_r)^{\beta}}{\sum_s \tau_s^{\alpha}(1/L_s)^{\beta}}.$$

Parámetros: diez hormigas por iteración, veinte iteraciones, $\alpha = 1$, $\beta = 2$, $\rho = 0,2$, $Q = 10$ y semilla `42`.

En cada iteración:

1. Calcule las probabilidades usando las feromonas actuales.
2. Las diez hormigas eligen rutas con esas mismas probabilidades.
3. Evapore la feromona de todas las rutas.
4. Cada hormiga deposita Q/L_r en la ruta elegida.

Entregue: código, frecuencias de elección, feromonas finales y mejor ruta observada.

Pregunta de interpretación: explique cómo la calidad de las rutas y la experiencia colectiva influyen en las elecciones posteriores.

Punto 5. PSO: movimiento de partículas y memoria — 1,0

Se desea minimizar $f(x) = x^2$. Una partícula tiene:

$$x = 4, \quad v = -1, \quad p_{\text{best}} = 2, \quad g_{\text{best}} = 1.$$

La actualización es:

$$v_{\text{nuevo}} = wv + c_1 r_1 (p_{\text{best}} - x) + c_2 r_2 (g_{\text{best}} - x),$$
$$x_{\text{nuevo}} = x + v_{\text{nuevo}}.$$

Use:

$$w = 0,5, \qquad c_1 = c_2 = 1,5, \qquad r_1 = 0,4, \qquad r_2 = 0,6.$$

A. Componente matemático y conceptual — 0,5

1. Calcule por separado el término de inercia, el cognitivo y el social.
2. Calcule la nueva velocidad, la nueva posición y su aptitud.
3. Determine si deben actualizarse p_{best} y g_{best}, suponiendo que las demás partículas no han mejorado.
4. Explique la diferencia entre la mejor posición personal y la mejor posición global.
5. Compare el intercambio de información en PSO con el de ACO.

B. Implementación en Python — 0,5

Implemente PSO para minimizar:

$$f(x, y) = x^2 + y^2, \qquad (x, y) \in [-5, 5]^2.$$

Use:

- Diez partículas.
- Posiciones iniciales uniformes en $[-5, 5]^2$.
- Velocidades iniciales iguales a cero.

- $w = 0,5$, $c_1 = c_2 = 1,5$.
- Nuevos valores r_1, r_2 independientes por partícula, dimensión e iteración.
- Treinta iteraciones y semilla `42`.
- Si una coordenada excede el dominio, ajústela al límite correspondiente y ponga en cero esa componente de velocidad.

Para cada iteración, use el mismo g_{best} al actualizar todas las partículas; después evalúe y actualice las memorias personales y global.

Entregue: código, mejor posición final, valor de la función e historial de la mejor aptitud global.

Pregunta de interpretación: ¿por qué las partículas pueden converger sin alcanzar exactamente $(0, 0)$?